

Semantic Structure and Revision in Temporal Evidence Graphs for Generative Monitoring

Anonymous Authors

Anonymous Institution

Abstract. Generated explanations can remain plausible after their evidence changes. We study a typed temporal property graph that records observations, proposed propositions, declared prerequisites and support at each knowledge time. A query-specific semantic core retains numerical witnesses and version history. A shared network view lets readers inspect original citations, corrected versions and alternative support routes in the paper and dashboard. Two conditional results bound which claims may change and characterize when direct maintenance misses required updates. We measure these misses through transitive exposure, the fraction of necessary changes excluded by direct scheduling. In synthetic, WESAD and PPG-DaLiA experiments, only 18 of 1201 displayed claims declare parents. Every eligible waveform revision has zero exposure. Independent scoring separates source grounding from task completeness. Checked displays are fully grounded, but subject-averaged required recall is 75.5–78.2%. Across 240 controlled structural cases, exact and GPU-generated programs expose necessary intermediate dependencies and surviving alternatives. Exposure predicts direct-maintenance misses in all 860 eligible replays. Full maintenance agrees across Neo4j and an indexed relational control. Generated answers reach evidence-to-answer depth one or two despite requests up to four. These results explain when explicit semantic structure enables selective revision. Storage size, path length and contract acceptance alone do not establish explanation fidelity or maintenance benefit.

Keywords: Temporal knowledge graphs · Evidence provenance · Generative explanations · Incremental maintenance · Physiological streams

1 Introduction

A generated explanation can become misleading when a sensor window is corrected, a feature fails a quality check, or a delayed observation replaces a statement of absence. Deleting the answer discards useful information. Keeping it may preserve a wrong number. A temporal evidence graph records what the answer asserted, which observations and prerequisites it cited, and what was known when.

A comparison may cite both a parent claim and the two operands directly. Revising either operand then reaches the comparison in one step. If an answer relies only on an intermediate claim, however, a direct checker may update that

claim and leave the answer unchanged. A reached claim may also remain supported because another valid route survives. Understanding these cases requires the meaning of the edges and support rules, not visual complexity.

We study a typed temporal property graph whose schema is specified by the researcher and whose instances record provenance and generated propositions. First, we define how the graph is constructed and how a query selects a semantic core linked to the stored objects. A shared network view lets a reviewer inspect original citations, version changes and groups of supporting inputs in the paper and an evidence-review dashboard. Second, we connect an explicit support model to a locality result and a measurable condition under which direct and transitive maintenance agree. Third, we test that connection using actual Neo4j exports, independent proposition scores and a controlled structural extension of three completed benchmarks. The graph records commitments made by the answer, not the model’s internal neural reasoning.

The waveform study provides a useful contrast. Only 18 of 1201 claims displayed by transitive graph maintenance (M1) declare parents, and direct checking matches transitive maintenance. Every eligible waveform revision batch has zero transitive exposure. A separate scoring protocol distinguishes accurate background statements from violations of the original target-only output contract. It reveals how checking trades coverage of required facts for grounded content. We then vary dependency depth and alternative support routes in exact numerical programs and common GPU-generated candidates over the same recordings. Together, these results show when propagation matters and when direct maintenance suffices.

1.1 Relation to Prior Work

Truth-maintenance systems revise justified beliefs when their premises change [1]. Incremental view maintenance updates database results when their inputs change [4]. Provenance semirings distinguish jointly required inputs from alternative derivations [3]. We connect these established ideas to the dependencies a bounded generator actually expresses and to independently measured maintenance outcomes.

PROV-DM distinguishes entities, activities, derivation, revision and attribution [13]. It informs our separation of recording provenance, generated assertions and review events. We implement a property graph with Python admission checks and database identity constraints. GraphRAG builds entity graphs and community summaries for corpus-level question answering [2]. Graphiti, described in the Zep architecture, maintains temporal information for agent memory [6]. Our unit of evaluation is a revisable numerical proposition and its admissible supporting inputs. We measure whether evidence grounds that proposition. This differs from asking whether a model’s account of its own computation faithfully describes its decision process [11].

2 A Typed Temporal Semantic Representation

2.1 Schema-Guided Ontology Instantiation

The ontology is fixed in code and specifies the allowed objects and relationships. The generator proposes instances within this vocabulary. Formally, let

$$\begin{aligned}\Omega &= (\mathcal{T}, \mathcal{R}, \text{dom}, \text{ran}, \mathcal{C}), \\ G_t &= (V_t, E_t, \vartheta_V, \vartheta_E, a_t).\end{aligned}\tag{1}$$

Here $\mathcal{T}$ and $\mathcal{R}$ are the node and relation vocabularies. $\vartheta_V : V_t \rightarrow \mathcal{T}$ and $\vartheta_E : E_t \rightarrow \mathcal{R}$ assign their types. An edge of type r connects allowed types in $\text{dom}(r)$ and $\text{ran}(r)$. Attributes a_t record immutable and logical IDs, dataset, subject, session, event interval, ingestion time, version, value, unit and evidence state. Constraints $\mathcal{C}$ require immutable identifiers, finite acyclic declared dependencies and the admission checks below. Ingestion time τ_v is distinct from the type maps ϑ . At knowledge time t , only records with $\tau_v \leq t$ are visible. Assessments become visible at their recorded knowledge time. Queries select the greatest version visible at their knowledge time, using ingestion time and ID to break ties. Historical queries retain that time. Tied assessments retain their trigger IDs. Conflicting states remain ambiguous because the timestamp does not establish the order of triggers.

Table 1 lists the stored objects. Observation windows carry hashes, sample indices and channel metadata, while raw arrays remain outside the database. Features carry units, extractor metadata and source IDs. The `with_entities` function adds subject and channel identities. The model proposes proposition fields, evidence references, parent links and sentences. Claims that pass the bounded checks are inserted immutably. The Neo4j adapter adds `SupportAssessment` nodes and `ASSESSED_AS` edges to the storage totals. Inserting an edge can create a placeholder for an unresolved source Record. None occurs in the original exports. Table 2 gives relation meanings and creation rules. Neo4j enforces unique scope/ID pairs. Python checks record kinds, immutable payloads, acyclicity and numerical admission. The database itself does not enforce these semantic checks.

Construction first normalizes the available evidence, then checks the model’s proposals and inserts the accepted records:

$$G_t = \Gamma_\Omega(E_{\leq t}, \hat{C}_{\leq t}) = \text{Insert}_{\text{immutable}}\left(\text{Check}_q\left(\text{Normalize}_\Omega(E_{\leq t}), \hat{C}_{\leq t}\right)\right).\tag{2}$$

$E_{\leq t}$ is the observation and revision log. $\hat{C}_{\leq t}$ contains proposals generated from the normalized visible evidence. Rejected proposals remain in the request archive. New assessments and explanation versions extend the graph without overwriting earlier content. The process instantiates the specified ontology rather than discovering one.

Table 1. Implemented object vocabulary. Checked status records an admission or assessment operation. It does not establish physiological truth.

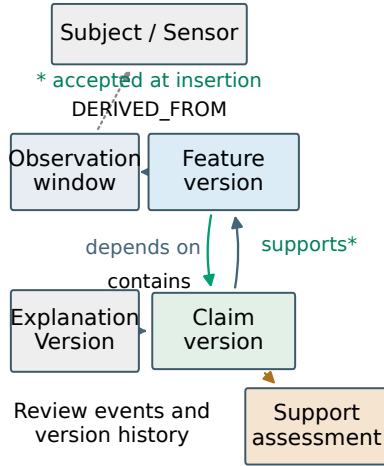
Stored object	Meaning	Construction
Subject / Sensor	Identity and channel	deterministic
Observation	Hashed sample window	deterministic
FeatureVersion	Value or availability watch	deterministic
ClaimVersion	Bounded proposition, references	model; checked
ExplanationVersion	Text and display membership	administrative
ReviewEvent	Recorded review action	administrative
SupportAssessment	State, time and trigger	evaluation
Record placeholder	Unresolved referenced ID	administrative

Table 2. Relation construction and direction. R denotes a Record, P an unresolved placeholder, F a feature, C a claim, O an observation, X an explanation and A an assessment. The adapter constructs edges. Admission checks give them claim-specific semantics. Administrative relations do not express entailment.

Relation	Domain $\rightarrow$ range	Meaning and creation rule
DEPENDS_ON	$R \rightarrow R/P$	Each source ID: deterministic input, model citation or declared parent.
DERIVED_FROM	$F \rightarrow R/P$	Feature source IDs: extraction provenance.
SUPPORTS	$R/P \rightarrow C$	Reverse source edge on admission; validity at insertion only.
SUPERSEDES	$R \rightarrow R/P$	New version to its predecessor ID.
CONTAINS	$X \rightarrow R/P$	Explanation source IDs: display membership; also dependencies.
ASSESSED_AS	$R \rightarrow A$	Outcome keyed by scope, record, knowledge time and trigger.
CONTRADICTS	$R \rightarrow R$	Trigger to adversely assessed record; time from assessment.
FOR_SUBJECT	$R \rightarrow \text{subject}$	Subject membership from deterministic identities.
OBSERVED_BY	$O \rightarrow \text{sensor}$	Channel identity from recording metadata.
REVIEWED_BY_EVENT	$R/P \rightarrow \text{review}$	Reviewed item to appended review event.

These rules explain much of the graph’s topology. Each `source_ids` entry creates a `DEPENDS_ON` edge from dependent to input. Features also receive `DERIVED_FROM` edges, and accepted claims receive `SUPPORTS` edges in the opposite direction. Display membership creates both `CONTAINS` and `DEPENDS_ON` edges. Subject membership creates `FOR_SUBJECT` hubs. Our analysis separates these administrative, provenance and reasoning roles.

a Fixed types and typed relations



b Saved WESAD S11 construction

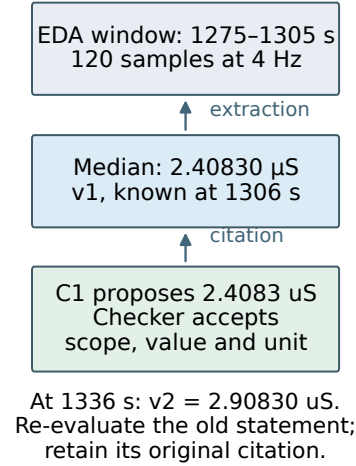


Fig. 1. Fixed vocabulary and an actual instance. Blue denotes deterministic recording and feature construction, green a checked model proposition, grey identity or display administration, and amber assessment or version effects. The concrete path comes from the exported WESAD S11 correction scope. Its source hash, sample indices and full IDs accompany the figure. Arrows preserve stored direction. The schema shown is a readable subset of the complete construction table.

In the saved WESAD S11 episode (Figure 1), the electrodermal activity (EDA) window at 1275–1305 s produces a median of 2.408303 μS . The model proposes 2.408303 μS and cites that feature. The claim passes the bounded checks. Insertion records its `DEPENDS_ON` citation and reverse `SUPPORTS` edge. A later feature version changes the median to 2.908303 μS , while the observation hash and original claim citation remain unchanged. Re-evaluation changes the claim’s current support without rewriting its history.

2.2 Edges as Explicit Semantic Commitments

An edge’s type determines what it asserts. A citation names evidence offered as support. A parent link declares a prerequisite. An extraction edge records how

a feature was produced. Membership and co-occurrence imply neither support nor a prerequisite. In particular, an LLM-proposed `DEPENDS_ON` edge does not prove that its target is necessary. `SUPPORTS` records acceptance when a claim was inserted, not validity at every later time. `CONTRADICTS` records an adverse assessment triggered by a revision, whose knowledge time is stored in the associated assessment. Exports retain these origins and distinguish proposed, accepted and displayed links.

2.3 The Query-Specific Semantic Core

To interpret and revise an answer, a reviewer needs its claims and the inputs they cite. We select this semantic core by a fixed projection:

$$K_{q,t} = \pi_{q,\Omega}(G_t), \quad V(K_{q,t}) = C_{q,t}^* \cup F_{q,t}, \quad E(K_{q,t}) = D_{q,t} \cup P_{q,t}. \quad (3)$$

$C_{q,t}^*$ contains the latest displayed answer’s claims and their declared claim ancestors. $F_{q,t}$ contains the visible feature propositions they cite, including explicit availability watches. $D_{q,t}$ retains the corresponding stored dependency edges. When a cited feature has a newer visible version, $P_{q,t}$ records a labelled path from the original citation through the version chain. Each such path keeps the citation and supersession witness IDs. It is never drawn as a directly stored Neo4j relationship.

Linked metadata retains subject and sensor identities, explanation containers, assessments and old versions. It also retains feature-to-observation provenance, which can be expanded in the drawing. A claim’s asserted fields determine its scope, not the wrapper’s query interval. Full IDs, raw sentences, original versions and assessment times remain recoverable. The core preserves the answer’s explicit commitments through this selection rule.

We analyze three distinct representations. The complete graph contains all stored objects. The active projection at t contains the latest evidence and current display members across queries, with their observation provenance. The core $K_{q,t}$ answers focused questions. Which window supports a value, and which operands support a comparison? Which version was known, what changed after correction, and does another witness still support the claim? We evaluate these competency questions against immutable input logs or exact program specifications, independently of database support flags.

2.4 Producing the Human-Review Network View

The review view helps a reader trace a selected answer or claim back to its evidence. It expands the semantic core with original cited versions, required observation provenance and visible successor versions. Citations stay attached to their original inputs. For a stored-node budget B and selected recorded times T , the transformation is

$$H_{q,t} = \text{View}_B(G_t, q), \quad p = \text{Layout}\left(\bigcup_{u \in T} H_{q,u}; m\right), \quad I_{q,t} = \text{Render}(H_{q,t}, p). \quad (4)$$

H is the bounded review expansion of K . The layout p stores node coordinates and routed edges, and rendering produces the drawing I . Mode m selects deterministic Graphviz `dot` for structured views or seeded spring positioning for the classic network. Spring positioning uses an undirected simple projection only to place nodes. Analysis and arrows retain the typed directed graph. Routing and collision separation change presentation, not support. Time comparisons reuse the layout of the union of their nodes. Records not yet ingested remain absent from the earlier drawing. Coordinates and distances do not measure semantic similarity or neural state.

The implementation selects a database scope, answer, focus and knowledge time, then follows the declared inputs. It expands observation provenance on request. Ingestion and assessment times determine visibility, while event intervals describe when the measurements or assertions apply. Original `DEPENDS_ON` edges remain intact. A separate derived path records how a citation resolves to a newer version, preserving the citation and the supersession edges traversed in reverse. Availability, assessed support and display membership remain separate. Among 432 assessment groups sharing a record and time, 245 have conflicting states. All outcomes and trigger IDs remain visible as ambiguous. Display membership follows the saved explanation version. Neither flags nor drawings determine the evaluator’s reference truth.

The node budget favors complete sets of alternative witnesses when they fit. Omitted records or incomplete witness groups produce a *partial view* warning and an expansion control. The view does not imply that omitted context is irrelevant. Subject and sensor hubs, display containers, assessments and reverse `SUPPORTS` edges remain in details. Structured views group feature rows in labelled AND enclosures. Each row maps to a stored ID, and each dashed bundle lists the original dependency edges it summarizes. These are drawing constructs. All inputs within a witness are required, while different witnesses are alternatives.

Green, blue and grey fills identify claims, features and observations. Outlines and words identify revisions, support and withdrawal. Every arrow is labelled with its stored direction or identified as a grouped edge. Figures 2 and 3 share their typed JSON views, ID maps and Graphviz renderer with the Streamlit dashboard. Its optional *Classic network* mode draws the same typed objects as individual ellipses in Figure 4. Selectboxes choose case, time, focus, witness route and node or edge. Details show full sentences, exact values and units, intervals, versions, source IDs and assessment provenance. Highlighting affects appearance only. Expansion reveals omitted provenance, and review actions append events in an isolated workspace. The interface makes external commitments inspectable. It does not establish a human-performance benefit or reveal neural reasoning.

3 Support, Locality and Transitive Exposure

3.1 Support Through Admissible Witnesses

A claim remains supported if at least one valid route still justifies it. For a proposition c , let $\mathcal{W}(c)$ be a finite family of admissible witnesses. Each witness W

groups evidence or prerequisite propositions that are jointly required. Different witnesses provide alternative routes:

$$\text{Supported}(c, t; q) = \bigvee_{W \in \mathcal{W}(c)} \left[\psi_c(W, t; q) \wedge \bigwedge_{u \in W} \text{Valid}(u, t; q) \right]. \quad (5)$$

A witness succeeds only when both checks pass. Valid requires observable availability and admissible temporal and subject scope. A prerequisite claim must itself be supported. The predicate ψ_c checks the asserted interval, selected logical versions, quantity, normalized unit and numerical operation. Numerical agreement uses the frozen tolerance $|v - \hat{v}| \leq \max(10^{-6}, 0.01|\hat{v}|)$. A correction can make ψ_c false while the source remains available. Historical and current assertions are different propositions. The extension’s specified inequalities use exact floating-point comparisons.

The original model supports local AND/OR composition and a two-operand difference. A generated numerical claim requires all its declared parents and an admissible direct numerical witness. Comparisons cite both operands. The extension represents alternatives as explicit lists of jointly required inputs, connected through an acyclic positive claim graph. It does not support unrestricted Boolean formulas. Each record’s semantic-program metadata stores the witness groups. Source IDs store their union for traversal, so edges alone do not recover the AND/OR structure. To claim that evidence is missing, the system needs a bounded set of possible producers and explicit availability watches. Absence from an open graph is insufficient.

As in provenance and truth maintenance [3,1], losing one route need not remove support if another admissible witness survives. Our alternatives use disjoint recording windows rather than renamed copies of one measurement. These distinct sources need not be statistically independent.

3.2 Locality of a Revision

Reachable claims need checking, but need not change. Let U_δ contain revised input identities, all earlier versions that can still be cited, and any changed availability subscriptions. Dependency edges point from a dependent to its input. Following them in reverse gives

$$R_\delta = U_\delta \cup \text{RevReach}(U_\delta), \quad B_\delta = \{c : S_{t-}(c) \neq S_{t+}(c)\}, \quad (6)$$

Here $S_t(c)$ abbreviates Equation 5. R_δ contains changed inputs and reachable dependents for re-evaluation. B_δ contains only claims whose support actually changes.

Proposition 1 (Conditional locality). *For a finite acyclic dependency model, $B_\delta \subseteq R_\delta$ when every mutable input to each support predicate is explicitly represented and only U_δ changes between the two snapshots.*

Proof sketch. Order nodes from inputs to dependents. A node outside R_δ has no changed input or prerequisite in R_δ , since either would make it reverse reachable. Its scope, numerical operands and availability are unchanged. Induction in this acyclic order gives unchanged prerequisite support and predicate values, hence unchanged support. Including earlier versions covers citations to v_1 after v_2 or v_3 arrives. Explicit availability records cover arrivals that falsify an absence statement. $\square$

Unrecorded clock expiry, newly discovered witnesses without subscriptions, or external predicate changes can violate locality. Our fixed-target queries and controlled witness sets represent changes through version or availability events. Benign updates and surviving alternatives show why the actual change set can be strictly smaller than the candidate set, $B_\delta \subset R_\delta$.

Indexed adjacency traversal costs $O(|V_{R_\delta}| + |E_{R_\delta}|)$. Current-version lookups, witness evaluation, snapshots, serialization, database round trips and writes add to this cost. The original runtime repeatedly builds version maps. Structural replay evaluates the whole small program before writing scheduled updates. Neo4j variable-length queries may enumerate paths, whereas SQLite uses a recursive SQL query with duplicate elimination. The abstract traversal bound therefore gives neither implementation a linear-time guarantee.

3.3 When Direct Maintenance Suffices

Direct maintenance can miss changes to unscheduled claims. Let A_δ be the independently determined set of prior admitted claims requiring a state change, and D_δ the claims actually scheduled by direct-only maintenance. For $|A_\delta| > 0$, the fraction outside that schedule is

$$\xi_\delta = \frac{|A_\delta \setminus D_\delta|}{|A_\delta|}. \quad (7)$$

We report NA when A_δ is empty. The primary measure concerns withdrawal of initially grounded claims. Restoration is recorded separately as a support transition. The event-log/task oracle determines A_δ independently of acceptance flags and graph paths.

Proposition 2 (Direct/full equivalence and misses). *Assume initially correct states, complete re-evaluation of D_δ , dependency semantics consistent with the oracle, and no unrelated refresh or other mechanism that revisits excluded claims. Before fresh generation, direct maintenance misses the fraction ξ_δ of required changes. A sufficient condition for direct and full maintenance to agree is that every changed support predicate is directly scheduled. In particular, this holds when each changed input affecting an answer has a direct dependency edge to every affected claim and all other predicates remain unchanged.*

Proof sketch. Complete direct re-evaluation changes every required member of D_δ . The no-refresh assumption leaves excluded members in their initial state. The missed set is therefore exactly $A_\delta \setminus D_\delta$. If all changed predicates are scheduled, both

algorithms evaluate the same predicates from the same snapshot. All remaining predicates retain equal states. $\square$

Consider $c_1 = \text{“test 1 holds”}$ and $c_2 = \text{“tests 1 and 2 hold”}$, where c_2 requires c_1 and test 2. Correcting test 1 to false requires both claims to change. A direct scheduler reaches only c_1 , so $\xi_\delta = 1/2$. If c_2 also cites test 1 directly, the scheduler reaches it and exposure is zero. If another independent route suffices for the answer, losing the first route need not change the answer at all. Path length alone therefore predicts neither a missed update nor a withdrawal. The waveform exports and controlled structural experiment test these different cases.

4 Evaluation Design

4.1 Preserved Three-Source Study

The unchanged `minimum_v1` study uses synthetic physiological-style waveforms, WESAD laboratory recordings [10,9], and PPG-DaLiA daily-activity recordings [8,7]. Each source contributes ten held-out subjects with two episodes each. The real-source splits use PCG64 seed 20260928 and five separate development subjects. Generation and checking exclude physiological and activity labels. Outcomes concern recorded features.

Each episode has a 30-second target interval and a preceding 120-second comparison interval. Four replay variants introduce clean or benign updates, channel faults, delays and numerical corrections. Three knowledge checkpoints occur at target end plus 1, 31 and 91 seconds. Four question families require the target EDA median plus its preceding-window difference, EDA missing fraction or acceleration variability. Requirements are fixed independently of answers. Five methods produce 3,600 initial cases. B0 displays unchecked proposals. B1 performs direct maintenance over indexed records, and B2 does so in Neo4j. M1 uses Neo4j with transitive maintenance. B3 implements the same transitive support rules in indexed SQLite with recursive common table expressions. All checked methods use the same fresh-answer checks and at most one repair.

The original study uses pinned Qwen3-8B BF16 [5], a three-claim contract and 1,536 output tokens. Both studies use the existing RTX 5090 and vLLM structured decoding [12], with model parameters and forward tensors verified on CUDA. Database operations and statistics run on the host CPU. Neo4j Community 5.26 and indexed SQLite retain their adapters. The three original B0 transport failures remain empty outputs.

4.2 Independent Semantic Evaluation

The `semantic_analysis_v1` oracle determines values and supporting witnesses from immutable events and task specifications. It uses neither support flags nor a neural verifier. Canonical proposition identity includes dataset, subject, session, asserted interval, quantity, operation, normalized value and unit, temporal mode and logical version. A correctly labelled background fact can be grounded without

answering the target question. Wrong numbers, subjects and target attributions still fail. Citation and witness fidelity are separate from directly checkable truth.

Precision asks how much emitted content is grounded. Recall asks how many answerable requirements it covers. For emitted checkable propositions $C(q, t)$ and answerable required propositions $R(q, t)$, we report

$$\begin{aligned} P_{\text{grounded}} &= \frac{\sum_{c \in C} \mathbf{1}[\text{source-grounded } c]}{|C|}, \\ R_{\text{required}} &= \frac{|\{r \in R : \text{matched by a grounded } c\}|}{|R|}. \end{aligned} \tag{8}$$

Each requirement is counted once, so repeated paraphrases cannot increase recall. The optional harmonic summary $S = 2P_{\text{grounded}}R_{\text{required}}/(P_{\text{grounded}} + R_{\text{required}})$ combines grounding and task completeness. It is not conventional set F1 because their universes differ. For an answerable question, empty output has precision NA and zero recall and summary score. Fully unanswerable questions have recall and summary NA, with absence statements checked separately. Partially answerable questions contribute their answerable facts.

We separately score raw and final proposals, accepted claims and displays. A limited prose audit retains raw text, literal discrepancies and interpretation terms. Structured scores do not certify unrestricted language. To assess temporal integrity, we score the same prior answer before arrival, after arrival and after maintenance. These discrete checkpoints do not establish a recovery curve. Withdrawing content and producing an adequate replacement remain separate outcomes.

4.3 Controlled Structural Extension

Before new outputs are observed, `structure_study_v1` selects the first five naturally sorted held-out subjects per source and both episodes. These 30 episodes reuse participants rather than forming a new independent cohort. Four intended evidence-to-answer depths (1–4) and two support regimes yield 240 cases. Each route compares EDA median, EDA interquartile range (IQR), acceleration standard deviation (SD) and EDA missing fraction with explicit bounds. The clean episode fixes these bounds so each route initially passes all four tests. The answer states whether at least one eligible window passes all tests. An alternative route uses the disjoint preceding window, which provides a distinct observation witness for that existential statement. The task makes no physiological diagnosis.

The depths group the same tests as direct inputs, a complete route followed by an answer, a two-test prefix followed by a route and answer, or a first test followed by that prefix, route and answer. Exact compiled fixtures instantiate these programs. Separately, the model proposes up to 16 claims and explicit witness lists, with 4,096 output tokens and at most one repair. The compiler never fills in missing model links. We retain requested and realized depth, omitted roots and incomplete alternative-witness families. Two development pilots precede the core freeze. The second uses a generic grouping example to clarify readable

sentences and reuse of emitted parent IDs. Neither pilot inserts required links. Both use development episodes, and their 40 calls are separate from the core budget.

Each method replays the same admitted candidate without generation after correction. B0 here disables maintenance on that content. Four independent update conditions make the first test false in every route, remove only the primary route, make a benign numerical change, or revise an unused cardiac feature. Neo4j and relational methods receive identical candidates, events and predicates. An independent truth-table evaluator enumerates the bounded Boolean assignments to test witness entailment and equivalence, allowing different valid derivations. A separate event-log evaluator determines required changes.

4.4 Graph Measurement and Uncertainty

Read-only Neo4j exports preserve nodes, relations, assessment times, Cypher and element IDs. Counts match the original ledger and sampled stored paths. Separate measurements of the full graph, active graph and query core cover type mixing, degree, depth, direct citations, witness counts, reachability and actual changes. Dependency degree excludes SUPPORTS and provenance mirrors. Meaningful zero categories remain, and we impose no power-law or embedding interpretation.

We first aggregate snapshots and variants within episodes, then episodes within subjects. Pointwise 95% intervals use 2,000 subject-cluster bootstrap resamples with seed 20260929. Historical versions and replay copies never count as independent participants. Detailed distributions and operational logs remain repository data. This paper retains the definitions and primary denominators needed to interpret the results.

5 Results

5.1 Storage Structure and Semantic Structure Differ

The export covers 480 retained Neo4j scopes. Counts match the original ledger, and 1850 representative stored paths are verified. M1 stores 47,038 nodes and 128,789 relations across its replay scopes. These totals measure storage, not independent observations. Membership and extraction rules create many of the edges. Neither these edges nor the reverse links added when claims are accepted imply deeper reasoning.

Across sources, the query core averages 2.8–4.1 nodes, or 1.5–2.2% of visible storage. It is smaller because it selects answer propositions and their declared inputs. The active graph also contains other visible evidence. Linked metadata preserves the observation witnesses. Full storage counts include historical versions, whereas core sizes count visible propositions for each query. Aggregating within subjects prevents replay copies from being treated as independent samples.

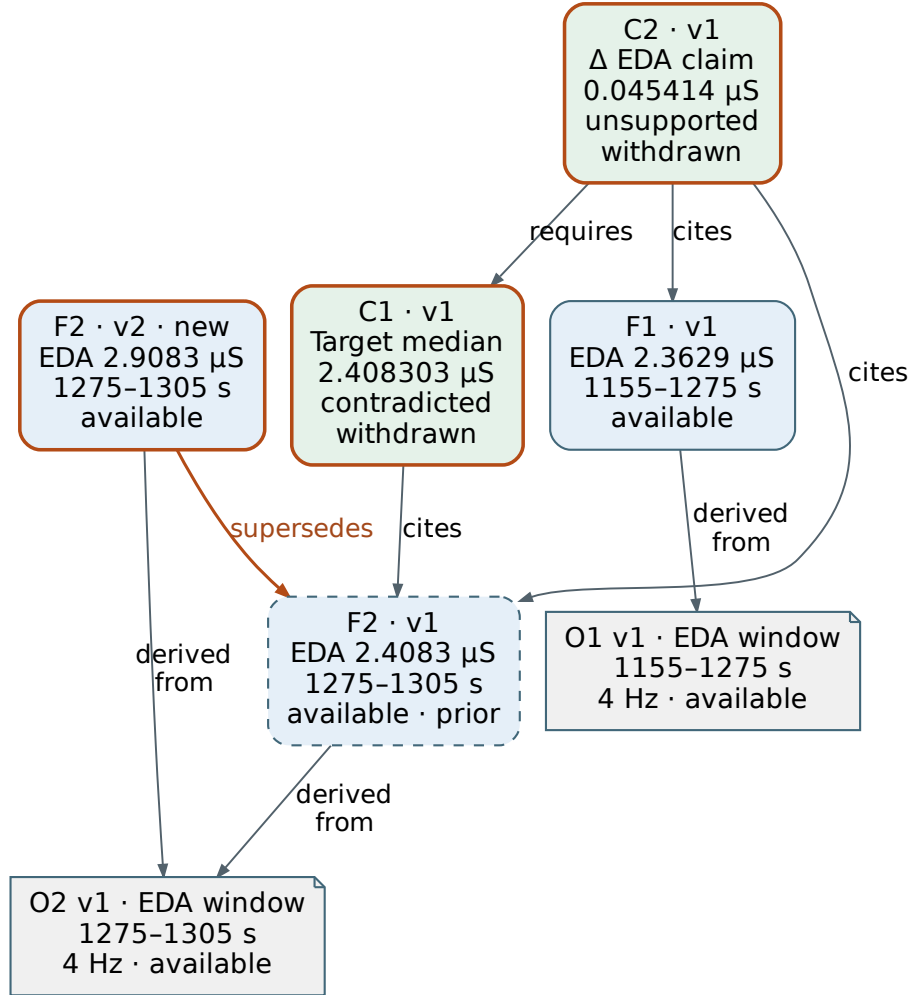


Fig. 2. Review network from actual Neo4j records for WESAD S11, episode 0, at knowledge time 1336.01 s. The unchanged median-size rule selects this case among 20 eligible correction cores. Green nodes are claims, blue nodes are features, and folded corners mark observations. *Cites/requires* mean **DEPENDS_ON**, *derived from* means **DERIVED_FROM**, and *supersedes* points from new to old. The injected correction adds F2 v2. C1/C2 retain their original v1 citations and are withdrawn. Availability remains distinct from current support. C2 cites both operands directly, so direct checking suffices. Identity hubs, admission mirrors and assessment/display containers are hidden but inspectable in details. Labels round values, while metadata preserves full precision.

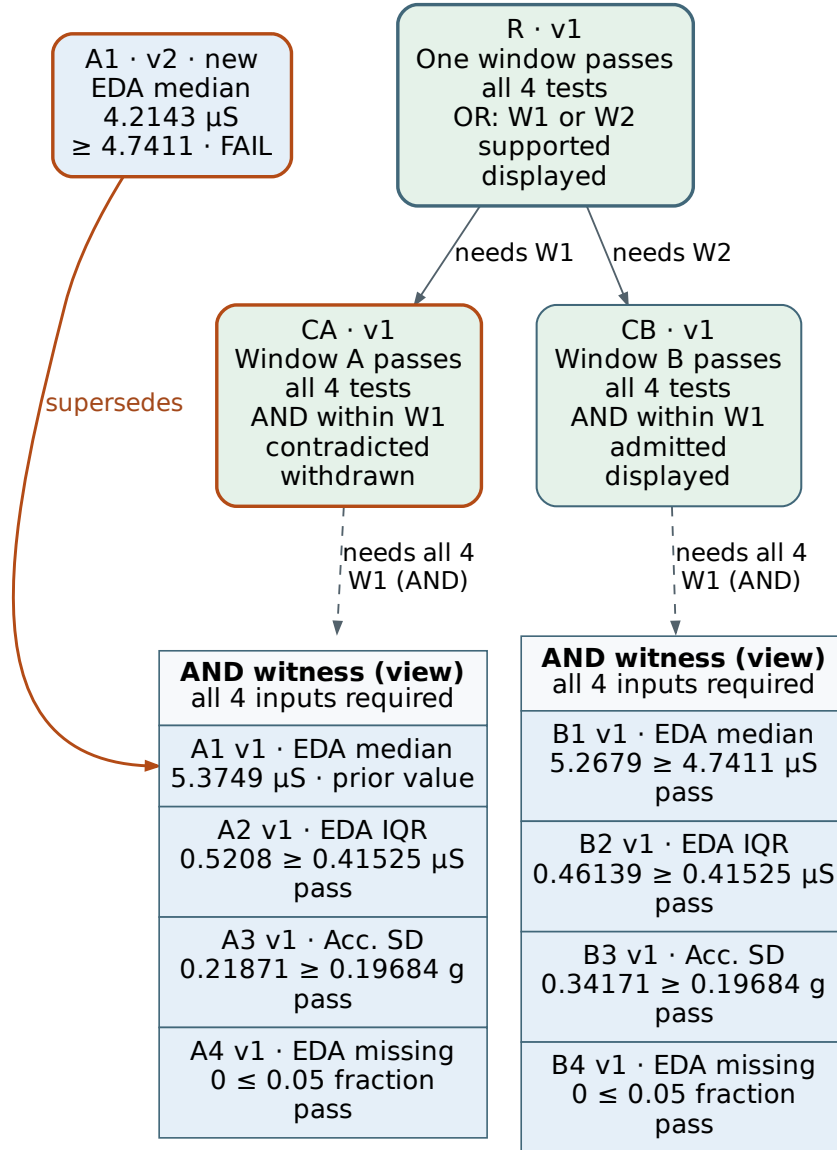


Fig. 3. Generated program from Neo4j for PPG-DaLiA S1 episode 0 at knowledge time 2331.01 s. Requested and realized depth are both 2. The smallest complete graph among 87 eligible programs is selected, with ties broken by ID. Windows A/B span 2270–2300/2150–2270 s. Correcting A1 invalidates CA, but CB still supports R. Alternative routes W1/W2 each require all four inputs. Blue rows are stored features. Dashed bundles summarize four `DEPENDS_ON` edges. AND enclosures are drawing constructs with exported row/edge mappings. Solid *needs* arrows are stored parent dependencies. Observation provenance expands on request. Administrative details remain inspectable.

Only 18 of 1201 displayed M1 claims declare a parent, leaving 98.5% parentless. Claim-parent depth never exceeds one. Even a parent edge need not create transitive exposure. In Figure 2, the comparison also cites the corrected feature directly. Of 360 revision batches, 210 require a withdrawal and 0 have nonzero exposure. The locality audit finds 0 violations. These dependencies explain why direct and full maintenance agree. The equivalence does not arise from missing stored paths or a particular database engine.

A worked ontology instance. Figure 4 shows how the ontology in Equations 1–2 represents the admitted PPG-DaLiA program. A1 is a particular **FeatureVersion** instance, not the feature class itself. The drawing shows 14 records and 18 stored relations from a scope of 26 objects and 83 relations. Its 11 starred records form the current core in Equation 3. CA remains a declared ancestor of R, although CA has been withdrawn from the displayed answer. Old A1 and the two EDA windows provide review context. Motion-window provenance can be expanded.

Each route requires four tests, and either route can support the answer. For this program, Equation 5 becomes

$$\begin{aligned} S(C_A, t) &= b_{A1}(t) \wedge b_{A2}(t) \wedge b_{A3}(t) \wedge b_{A4}(t) \\ S(C_B, t) &= b_{B1}(t) \wedge b_{B2}(t) \wedge b_{B3}(t) \wedge b_{B4}(t) \\ S(R, t) &= S(C_A, t) \vee S(C_B, t) \end{aligned}$$

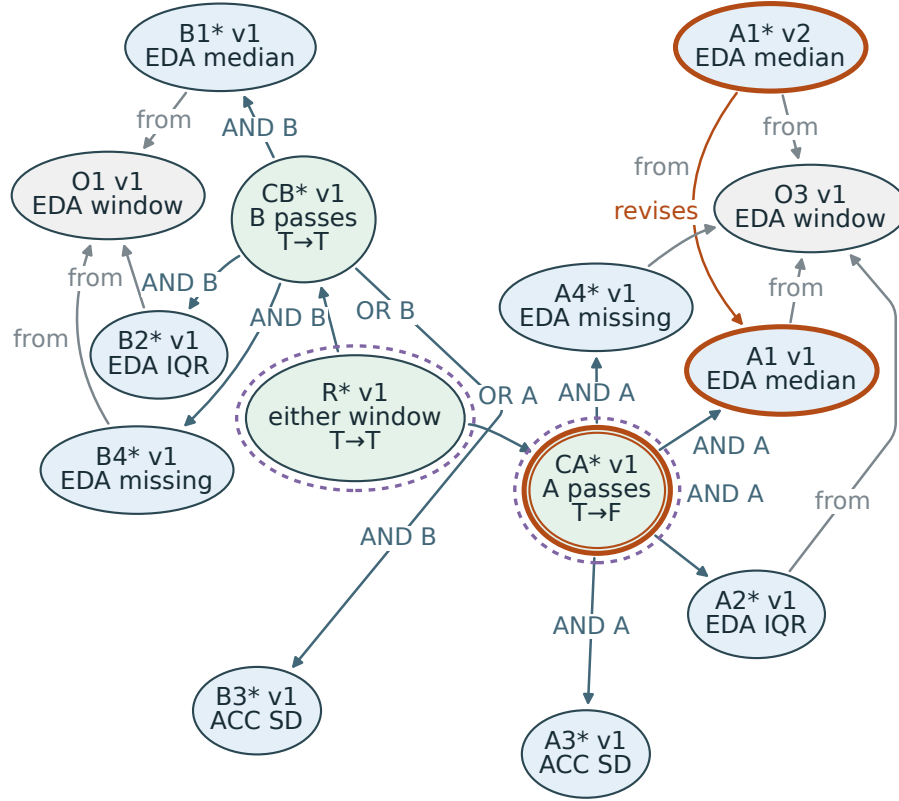
Each b_{r_i} checks the latest visible feature for PPG-DaLiA S1, session 1, with the specified A/B interval, quantity and unit and an available non-null value. It then applies the exact $\geq$ threshold for median, IQR and acceleration SD, or $\leq$ for missing fraction (Figure 3). A1 changes from 5.374856 μ S, above 4.74108975 μ S, to 4.214302 μ S, below it. The measurement remains available, but b_{A1} becomes false. The witness lists therefore give $(S(C_A), S(C_B), S(R)) : (1, 1, 1) \rightarrow (0, 1, 1)$.

The theorem’s dependency projection contains 13 nodes and 10 edges. Reachability excludes provenance, membership and support mirrors. Equation 6 gives $U_\delta = \{A1_{v1}, A1_{v2}\}$ and $R_\delta = U_\delta \cup \{C_A, R\}$, but $B_\delta = \{C_A\}$. These full computed sets are all visible here. The independent event/task oracle and recorded B2 schedule separately give $A_\delta = D_\delta = \{C_A\}$. Exposure in Equation 7 is $0/1 = 0$. Two dependent claims need re-evaluation, yet only CA changes support because another witness preserves the root. Direct and full maintenance agree.

5.2 Grounded Content and Useful Coverage

Emitted-claim counts before arrival, after arrival and after maintenance are Synthetic 211/211/138, WESAD 281/281/207, and PPG-DaLiA 271/271/191. Each source has 260/290/290 answerable required facts at these checkpoints. Precision excludes empty outputs, whereas recall includes their zero coverage. These pooled counts describe the sample. Reported percentages average episodes within subjects.

Checking produces fully grounded displays under the bounded numerical protocol, but it also removes useful information.



○ ClaimVersion ○ FeatureVersion ○ Observation

* in current core K; unstarred nodes are review context

Orange: input versions U_δ; dotted: reachable claims

Double outline: changed support B_δ; labels: before → after

Full claim sets: reachable 2; changed 1; $\xi\delta = 0/1 = 0$

Fig. 4. Classic ontology-instance view of the admitted program `ppg_dalia-S1-e0-d2-alternative`, PPG-DaLiA S1 episode 0. The primary route is corrected at 2331.0 s. The view compares 2331.01 s with 2301.01 s. All 14 ellipses are stored records. AND A/B identify the four jointly required inputs of CA/CB. OR A/B identify R's alternative parent witnesses. Both mean `DEPENDS_ON`. *From* means `DERIVED_FROM`, and *revises* means new-to-old `SUPERSEDES`. Original citations remain intact. Motion windows, administrative relations, feature dependency duplicates and `SUPPORTS` mirrors are hidden. Fill identifies ontology class. Outlines and stars show computed revision sets and core membership. Visual annotations add no database nodes or dependencies.

Table 3. Independent semantic scores in percent (subject means and 95% intervals). Each row contains 240 answer instances from 20 episodes. Matched counts unique requirements within each answer. Denominators count answerable requirements. Empty displays have zero recall on answerable questions and undefined precision. The original strict-contract interpretation is preserved separately from these scores.

Source	State	P [95% CI]	R [95% CI]	Matched	Empty
Synthetic	B0 display	87.8 [84.5, 90.5]	95.5 [90.0, 99.5]	402/420	1
Synthetic	M1 raw	87.3 [83.4, 90.4]	95.0 [88.5, 100.0]	400/420	0
Synthetic	M1 display	100.0 [100.0, 100.0]	78.2 [66.2, 90.0]	333/420	20
WESAD	B0 display	83.6 [77.2, 89.4]	86.5 [77.8, 95.0]	366/420	1
WESAD	M1 raw	83.4 [77.1, 89.2]	87.0 [78.2, 95.5]	368/420	0
WESAD	M1 display	100.0 [100.0, 100.0]	75.8 [64.0, 87.5]	323/420	20
PPG-DaLiA	B0 display	84.5 [78.7, 89.6]	87.8 [79.2, 94.8]	371/420	1
PPG-DaLiA	M1 raw	84.7 [79.2, 89.6]	88.8 [80.7, 95.5]	375/420	0
PPG-DaLiA	M1 display	100.0 [100.0, 100.0]	75.5 [65.0, 87.5]	322/420	20

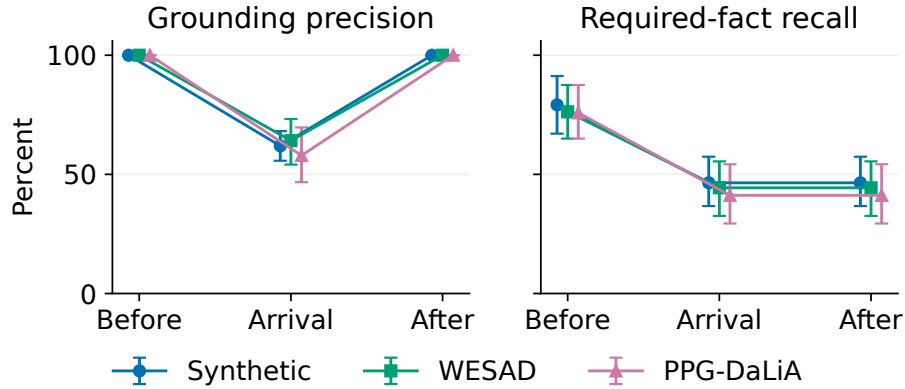


Fig. 5. Independent grounding precision and required recall for the same prior M1 answers at three discrete checkpoints, with subject-cluster 95% intervals. Each source has 160 transitions from 20 episodes and ten subjects across the four variants. No smooth recovery curve is inferred. Emitted-claim and required-fact denominators appear in the text. Fresh-answer scores remain in Table 3.

Raw M1 required recall is 95.0%, 87.0% and 88.8% for synthetic, WESAD and PPG-DaLiA respectively. Displayed recall falls to 78.2%, 75.8% and 75.5% (Table 3). Thus the precision gain partly comes from rejecting correct, useful propositions. Figure 5 follows the same prior answers. Withdrawing stale assertions restores grounding among retained claims, but required facts can remain unanswered.

Of 799 B0 claims rejected by the frozen contract, 499 are grounded under the new proposition protocol. This includes 333 correctly time-labelled background observations. Other disagreements arise when dependency or citation restrictions reject correct numerical content. No claim that passed the old contract becomes numerically ungrounded under the new scoring. A wrong target attribution still fails because matching uses the asserted interval and independently resolved version.

Queries about source windows and known versions agree with the independent log oracle on all 1201 feature witnesses. Queries about comparison operands agree on 18 eligible displayed comparisons. These checks recover admissible witnesses without requiring a unique gold reasoning graph. The separate prose audit and original auxiliary-verifier disagreements remain available. The bounded scores do not certify unrestricted interpretation.

5.3 Structural Variation Tests the Maintenance Condition

Table 4. Common-candidate structural replay. Counts aggregate four update conditions per case, separately for exact and generated programs. A required change is independently determined before scheduling. Correction intervals average episodes within subjects identified separately for each dataset. B1 matches B2 and B3 matches M1 in every cell. Missing generated roots are included in the depth/coverage analysis, not invented as initially displayed claims.

Program	Method	Required	Missed	Collateral	Corrected % [CI]
Exact	B0	1260	1260	0	0.0 [0.0, 0.0]
Exact	B2	1260	720	0	42.9 [42.9, 42.9]
Exact	M1	1260	0	0	100.0 [100.0, 100.0]
Generated	B0	1210	1210	0	0.0 [0.0, 0.0]
Generated	B2	1210	183	0	84.9 [84.0, 85.7]
Generated	M1	1210	0	0	100.0 [100.0, 100.0]

The extension produces 240 common candidates with 325 GPU calls, including 85 repairs, in 17.2 minutes. Its 9,600 replays include exact and generated programs. Neo4j and the relational control agree in all 1,920 matched groups of program origin and update. Direct maintenance misses 720 of 1260 required changes in exact programs and 183 of 1210 in admitted generated programs. Full maintenance misses none and withdraws no unaffected claims. Predicted misses agree in all 860 of 860 eligible groups (Figure 6).

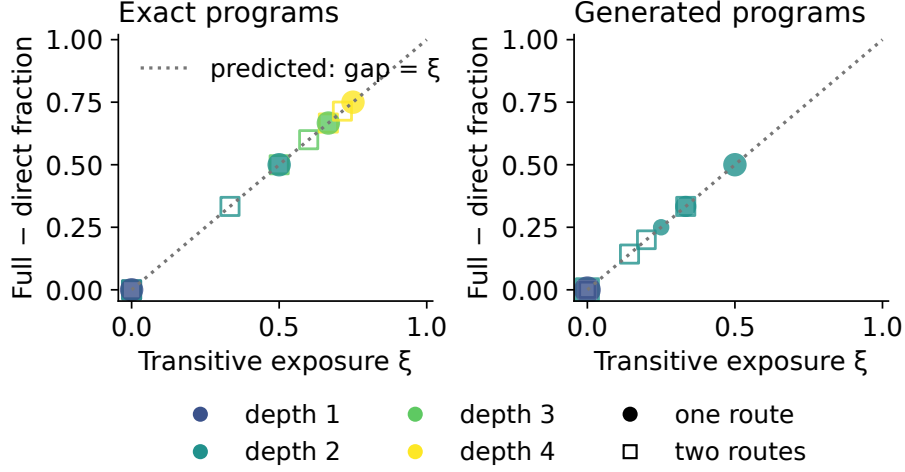


Fig. 6. Population-level structural test. Full-minus-direct correction fraction versus independently computed transitive exposure, separately for exact compiled and admitted GPU-generated programs. Points aggregate equal exposure, support regime and realized depth. Size reflects batch count, and the diagonal shows the conditional prediction. These are primary-admission results. The text reports the separate sensitivity analysis after normalization.

Generated answers have depth one in 74 cases and depth two in 135 cases. Another 31 cases have no admitted answer root. None reaches depth three or four. The model usually proposes route claims followed by an answer. Exact programs test deeper counterexamples, while generated programs test the expressed dependencies. When the primary route is lost, all 120 exact and 96 generated two-route answers with admitted roots remain supported. Single-route roots do not. In Figure 3, A1 changes from 5.3749 to 4.2143 μ S, below its 4.7411 μ S threshold. CA is withdrawn, but CB and the answer remain displayed. Direct and full maintenance agree for this update. Other tested conditions require updates to dependents excluded by direct scheduling. Necessary dependencies and surviving alternatives determine the outcome, not path length. In 236 paired cases, programs with the same evidence and baseline record count have different exposure. Storage size alone therefore cannot explain maintenance behavior.

After the core run, we found an omission in the admission table. Primitive test names were defined in the input but unavailable as claim proposition names. This produced 236 unknown-proposition rejection reasons. A separate corrected replication maps those names to their supplied comparators without changing candidates, witnesses or links. It admits 229 roots and repeats 4,800 CPU database replays. Direct maintenance misses 208 of 1294 required changes, while full maintenance again misses none. The original scores remain frozen. Truth-

table scoring independently distinguishes initially true statements from sound, complete witness families. This admission correction adds no model reasoning.

5.4 Operational Cost and Adequate Replacement

The original systems workload produces 0 adequate replacements among 236 completed candidates. All omit the required preceding-window difference, so no successful replacement time is observed. At the highest offered rate, both databases accumulate a backlog and delay flags. The extension maintains a common answer without post-correction generation. It establishes selective withdrawal and preservation, not an adequate replacement. Replay timings include diagnostic traversal and whole-program evaluation. They measure instrumented costs rather than production latency.

6 Discussion and Conclusion

The representation distinguishes a changed value, an invalid prerequisite and a surviving witness. The ontology defines these commitments, and the review view exposes their evidence and versions. Locality limits which claims need re-evaluation. Transitive exposure identifies required changes that direct scheduling excludes. The two measured regimes show why declared paths and requested depth alone do not establish a maintenance advantage.

Withdrawing stale content can preserve grounding while leaving required facts unanswered. A strict contract can also reject correct background information. Independent proposition and witness scores distinguish these outcomes. Admission normalization changes which supplied definitions the checker recognizes without changing model outputs or parent links.

The findings concern bounded propositions, fixed witness sets and acyclic dependencies. Agreement with recordings does not establish physiological truth. Natural faults, clock expiry, open-ended evidence discovery and unrestricted prose need further study. Generalization is limited by ten held-out subjects per source, five reused in the extension, one generator and one seed. Disjoint windows are not statistically independent. Neo4j and the relational control agree on matched updates, without establishing a universal ranking of storage performance. The dashboard supports event-driven inspection and isolated review actions. Human decision accuracy, interaction latency and real-time benefit remain untested. The original failure to produce adequate replacements limits recovery claims.

Artifacts. The repository preserves frozen studies, raw candidates, independent scores, database exports, shared review views, vector figures, reproduction commands and exhaustive logs. All essential definitions and sensitivity results appear here. Provider recordings are not redistributed.

Use of generative AI. An AI coding assistant supported implementation, analysis and drafting. Qwen3-8B generated candidates, and Qwen2.5-3B-Instruct supplied the auxiliary audit. Human authors must review the evidence and interpretations before submission.

References

1. Doyle, J.: A truth maintenance system. *Artificial Intelligence* **12**(3), 231–272 (1979). [https://doi.org/10.1016/0004-3702\(79\)90008-0](https://doi.org/10.1016/0004-3702(79)90008-0)
2. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R.O., Larson, J.: From local to global: A graph RAG approach to query-focused summarization (2025), <https://arxiv.org/abs/2404.16130v2>, version 2
3. Green, T.J., Karvounarakis, G., Tannen, V.: Provenance semirings. In: *Proceedings of ACM PODS*. pp. 31–40 (2007). <https://doi.org/10.1145/1265530.1265535>
4. Gupta, A., Mumick, I.S., Subrahmanian, V.S.: Maintaining views incrementally. In: *Proceedings of ACM SIGMOD*. pp. 157–166 (1993). <https://doi.org/10.1145/170035.170066>
5. Qwen Team: Qwen3-8B model card (2025), <https://huggingface.co/Qwen/Qwen3-8B>, revision b968826d9c46dd6066d109eabc6255188de91218
6. Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., Chalef, D.: Zep: A temporal knowledge graph architecture for agent memory (2025), <https://arxiv.org/abs/2501.13956>
7. Reiss, A., Indlekofer, I., Schmidt, P.: PPG-DaLiA. *UCI Machine Learning Repository* (2019). <https://doi.org/10.24432/C53890>
8. Reiss, A., Indlekofer, I., Schmidt, P., Van Laerhoven, K.: Deep PPG: Large-scale heart rate estimation with convolutional neural networks. *Sensors* **19**(14), 3079 (2019). <https://doi.org/10.3390/s19143079>
9. Schmidt, P., Reiss, A.: WESAD (wearable stress and affect detection). *UCI Machine Learning Repository* (2018). <https://doi.org/10.24432/C57K5T>
10. Schmidt, P., Reiss, A., Duerichen, R., Marberger, C., Van Laerhoven, K.: Introducing WESAD, a multimodal dataset for wearable stress and affect detection. In: *Proceedings of the International Conference on Multimodal Interaction*. pp. 400–408 (2018). <https://doi.org/10.1145/3242969.3242985>
11. Turpin, M., Michael, J., Perez, E., Bowman, S.R.: Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In: *Advances in Neural Information Processing Systems* (2023), <https://arxiv.org/abs/2305.04388>
12. vLLM Contributors: Structured outputs (2025), https://docs.vllm.ai/en/v0.10.2/features/structured_outputs.html, version 0.10.2
13. W3C PROV Working Group: PROV-DM: The PROV data model. W3C recommendation, World Wide Web Consortium (April 2013), <https://www.w3.org/TR/prov-dm/>